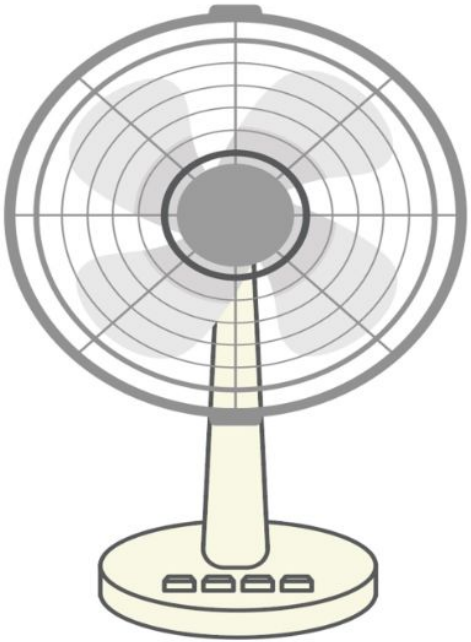
Pomodoro Breeze

박도영
김수연
이동준

Contents

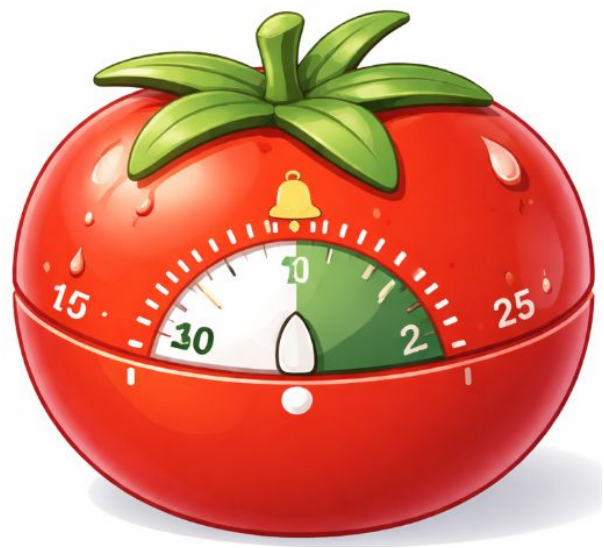


1. 프로젝트 주제
2. 시연 영상
3. 프로젝트 기획 및 운영
4. 시스템 구성
5. 상태 머신 및 동작 흐름
6. 핵심 구현 내용
7. 문제 해결 사례
8. 개선점 및 확장 아이디어

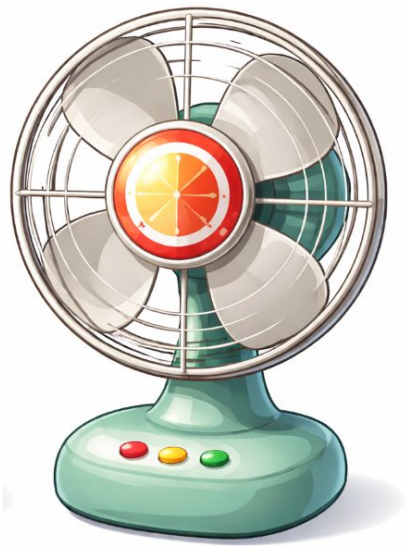
1. 프로젝트 주제 _ Pomodoro Breeze 소개

- 짧게 집중하고 쉬는 시간 관리 기법과 선풍기 동작을 결합한 집중 보조 시스템

<Pomodoro>

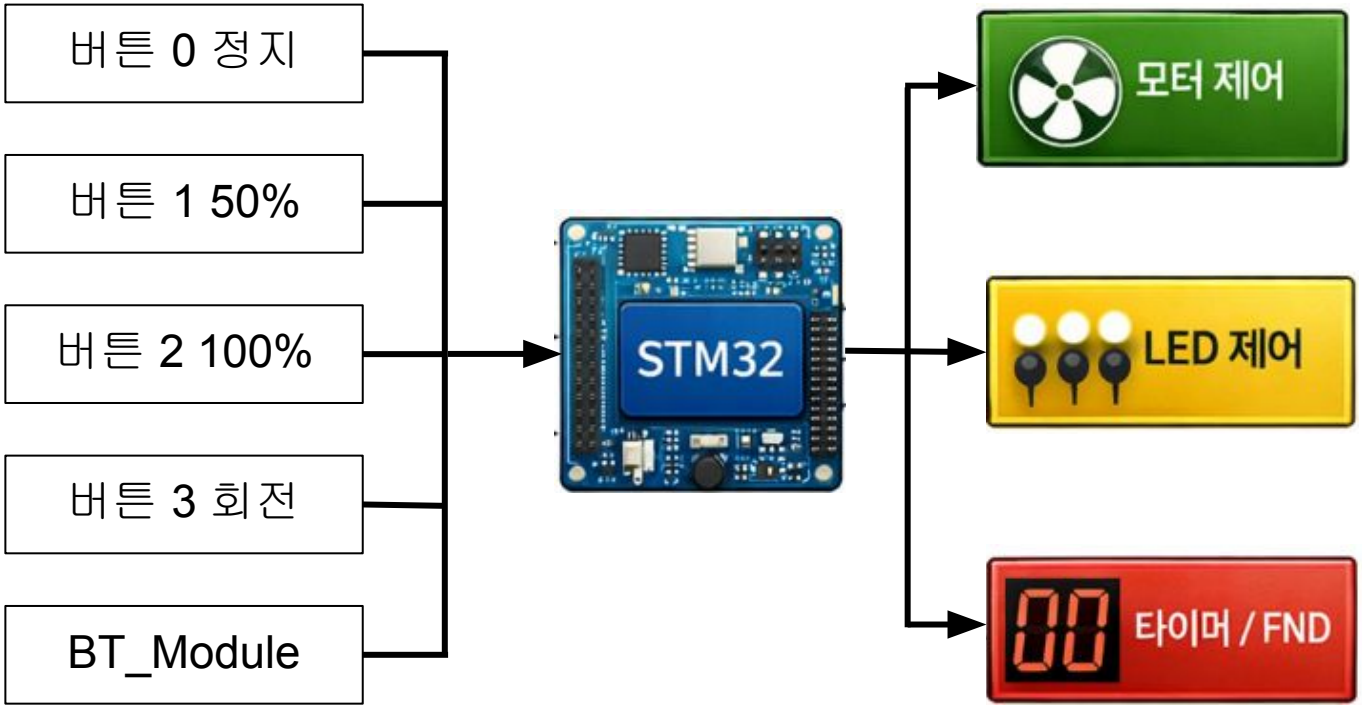


<Breeze>



1. 프로젝트 주제 _ Function

- 버튼 · BT · Fan · Servo · LED · FND 활용
- 타이머 표시 및 자동 종료 기능



2. 시연 영상 + Git repo.

❑ 시연 영상(Youtube)과 GIT 코드 공유

[Youtube]

https://youtube.com/shorts/TqgLxleKQTw?si=G4J6Op7MCj9wlX6H

[GIT]

3. 프로젝트 기획 및 운영 _ Team Members

[박도영]

- Button, Servo 제어
- UART 제어
- 내부 회로 구성
- 발표자료 제작
- 발표

[김수연]

- Fan 제어
- LED, FND 제어
- Timer 제어
- 발표자료 제작
- Git

[이동준]

- LED 제어
- 외형 제작

3. 프로젝트 기획 및 운영 _ Project Schedule

1/27
- 브레인스토밍
- Button 제어

1/28
- Servo 제어
- Fan 제어
- FND 제어

1/29
- LED 제어
- BT 제어
- 외형 구상

1/30
- 외형 제작
- Timer 제어
- 통합 제어

1/31
- 외형 제작
- 제작 후 테스트
- 버그 수정

2/1
- 시연 영상 촬영
- 발표자료 제작

2/2
- 발표

4. 시스템 구성 _ HW / SW / Equipment

stm411 Board 1EA

Push Button 4Ea

FND(anode) 1EA

Fan 1EA

LED 1EA

BT Module 1Ea

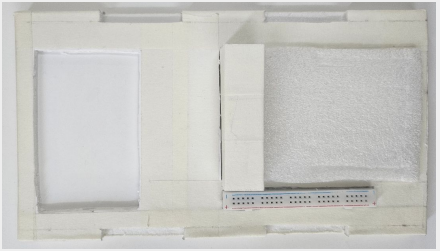
Servo Motor 1EA

STM32 CubeIDE

STM32 CubeMX

4. 시스템 구성 _ 조립도

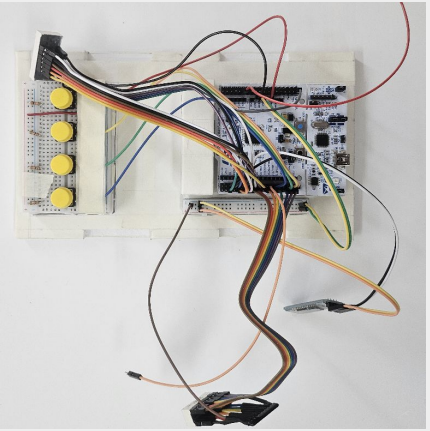
[Bottom Cover]

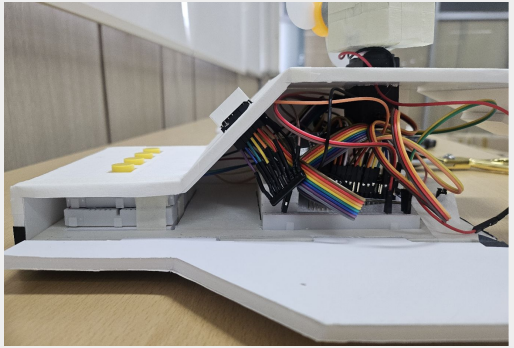


[Side Cover]

[Top Cover]

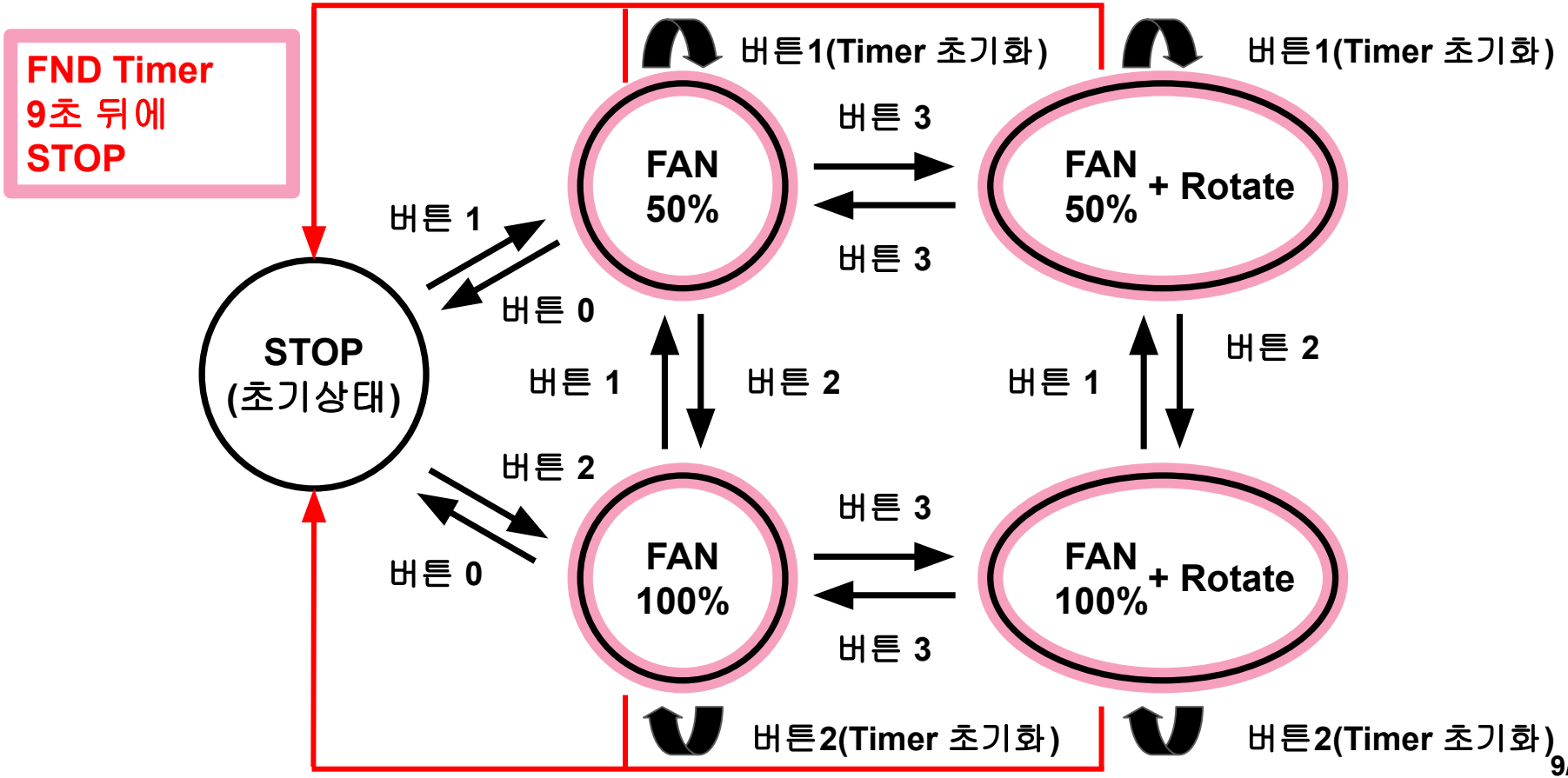
[Wiring & Assembly]





[Pomodoro Breeze]

5. 상태 머신 및 동작 흐름


FND Timer
9초 뒤에
STOP
버튼1(Timer 초기화)
버튼1(Timer 초기화)
FAN
50%
버튼 3
FAN
50%
+ Rotate
버튼 3
버튼 1
버튼 0
STOP
(초기상태)
버튼 1
버튼 2
버튼 1
버튼 2
버튼 2
버튼 0
FAN
100%
버튼 3
FAN
100%
+ Rotate
버튼 3
버튼2(Timer 초기화)
버튼2(Timer 초기화)

6. 핵심 구현 내용 _ Servo Motor

(1) Servo Motor

```c
// main 함수 호출용
void Servo_Run(void)
{
  if (servoState == SERVO_STOP)        return;


  /* 20ms마다 한 칸 이동 */
  if (HAL_GetTick() - prevTime < 20)  return;

  prevTime = HAL_GetTick();

  if (direction == 1)
  {
      Servo_Position = Servo_Position + 1;
  }
  else
  {
      Servo_Position = Servo_Position - 1;
  }

  if (Servo_Position >= SERVO_RIGHT)
  {
      Servo_Position = SERVO_RIGHT;
      direction = 0;
  }

  if (Servo_Position <= SERVO_LEFT)
  {
      Servo_Position = SERVO_LEFT;
      direction = 1;
  }

  TIM3->CCR1 = Servo_Position;
}
```

[Design Goal]

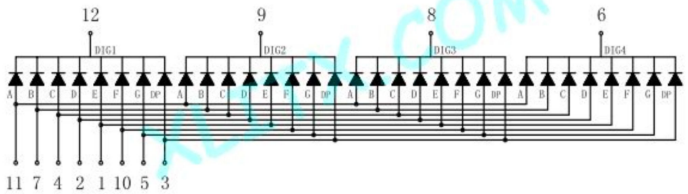
- 서보모터를 **PWM 기반**으로 왕복 회전 제어
- tick 기반 일정 속도의 non-blocking 동작 구현

[Implementation Summary]

1. 구조체로 서보 상태 관리
   - state, position, time
2. Timer/Counter
   - timer 1 이용
   - PSC 1000, AGG 2000 사용
3. prevTime으로 서보 위치 갱신
4. 20ms마다 position을 1step씩 변경해 회전

6. 핵심 구현 내용 _ FND

(2) FND




```c
void fndDisplay(uint16_t data)
{
    static uint8_t position = 0;
    uint8_t value;

    uint8_t sec = data / 100;   //
    uint8_t ms  = data % 100;   //

    /* digit OFF */
    fndAllDigitOff();

    /* segment OFF (Anode) */
    fndWriteSegment(0x00);
    fndDotOff();

    /* 자리별 값 선택 */
    switch (position)
    {
        case 0: value = sec / 10; break;   // 초 tens
        case 1: value = sec % 10; break;   // 초 ones
        case 2: value = ms  / 10; break;   // ms tens
        case 3: value = ms  % 10; break;   // ms ones
    }

    /* 세그먼트 출력 */
    fndWriteSegment(fndData[value]);

    if (position == 1)
        {
            fndDotOn();
        }

    /* 자리 선택 */
    switch (position)
    {
        case 0: HAL_GPIO_WritePin(GPIOB, GPIO_PIN_15, GPIO_PIN_SET); break;
        case 1: HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, GPIO_PIN_SET); break;
        case 2: HAL_GPIO_WritePin(GPIOB, GPIO_PIN_13, GPIO_PIN_SET); break;
        case 3: HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4,  GPIO_PIN_SET); break;
    }

    position = (position + 1) % 4;
}
```

[Design Goal]

- 4-digit FND display (Anode)
- **sec/ms** 데이터 시각화 **, dot** 제어

[Implementation Summary]

1. Segment data pin mapping
2. Display control
   - segment GPIO 출력
3. Display update
   - position 순환으로 4-digit 출력
   - dot으로 sec/ms 분리 표시

6. 핵심 구현 내용 _ LED

(3) LED

```c
void ledLeftShiftFan_Task(uint8_t num)
{
    static int8_t direction = 1; // 1이면 증가, -1이면 감소

    if (!fanRun) return;

    if (HAL_GetTick() - fanTick >= 200)
    {
        fanTick = HAL_GetTick();

        // 1. 모든 LED를 먼저 off
        ledOff(8);

        // 2. 현재 위치의 LED만on
        ledOn(fanIndex);

        // 3. 방향에 따라 인덱스 변경
        fanIndex += direction;

        // 4. 끝에 도달하면 방향 반전
        // num이 8이라면: 8에 도달하는 순간 다음은 7이 되도록 설정
        if (fanIndex >= num)
        {
            fanIndex = num;
            direction = -1;
        }
        else if (fanIndex <= 0)
        {
            fanIndex = 0;
            direction = 1;
        }
    }
}
```

[Design Goal]

- **LED를 fan(왕복 shift) 패턴으로 구동**
- timer 기반 non-blocking LED animation 구현

[Implementation Summary]

1. LED 정보 구조체 배열로 관리
   - port, pin, on/off state
2. Left/Right shift
   - tick 기반으로 LED index 이동
   - 끝 도달 시 방향 반전

6. 핵심 구현 내용 _ App

(4.1) App

```c
void App_Init(void)
{
    ledOff(8);
}

void App_HandleStop(void)
{
    Servo_Stop();
    TIM2->CCR1 = 0;
    runFlag = 0;
    ledCount = 0;
    ledOff(8);
    ledLeftShiftFan_Stop();
    rotateFlag = 0;
}
```

```c
void App_Handle50(void)
{
    TIM2->CCR1 = 1000;
    startTick = HAL_GetTick();
    runFlag = 1;
    ledCount = 4;
    ledOff(8);
    ledOn(4);
    rotateFlag = 1;
}

void App_Handle100(void)
{
    TIM2->CCR1 = 2000;
    startTick = HAL_GetTick();
    runFlag = 1;
    ledCount = 8;
    ledOff(8);
    ledOn(8);
    rotateFlag = 1;
}
```

[Design Goal]

- **Button/UART 입력에 따른 시스템 동작 제어**
- **Fan/Servo/LED/FND 동작을 상태 기반으로 통합 관리**

[Implementation Summary]

- 팬의 상태를 나타내는 함수로 리팩토링
  - PWM(CCR)으로 Fan 강도 제어
  - LED/Servo 함수 call
  - UART command로 동일 기능

6. 핵심 구현 내용 _ App

(4.2) App

```c
void App_Task(void)
{
    uint16_t display = 0;

    if (buttonGetPressed(0))
    {
      App_HandleStop();
    }
    else if (buttonGetPressed(1)){
      App_Handle50();
    }
    else if (buttonGetPressed(2))
    {
      App_Handle100();
    }


    if (rotateFlag)
    {
      if (buttonGetPressed(3))
      {
        App_HandleServo();
      }
    }

    Servo_Run();

    if (Servo_IsRunning())
        ledLeftShiftFan_Task(ledCount);

    if (runFlag)
    {
        uint32_t now = HAL_GetTick() - startTick;
        if (now >= 9000)
        {
          App_HandleStop();
        }
        else
            display = ((now / 1000) % 100) * 100 + (now % 1000) / 10;
    }

    fndDisplay(display);
}
```

[Implementation Summary]

1. Input handling
   - button 입력으로 Stop/50%/100%/Servo 회전
   - Button 0 : STOP
   - Button 1 : Fan 50%
   - Button 2 : Fan 100%
   - Button 3 : Servo rotate
     - 정지상태에서 누르면 동작 X
     - 동작상태에서 누르면 회전 종료
2. 9초후 자동 정지

7. 문제 해결 사례

Problem 1. LED Shift issue

회전버튼을 눌렀을 때 LED만 동작하고, 다른 Component (FND, Servo 등)들이 전부 멈춤

Solution 1. Hal_delay 대신 Tick 사용

원인 : Hal_delay함수때문에 CPU 점유 -> Tick 함수를 이용해 non-blocking 방식으로 구현

Problem 2. Button 눌림 issue

Button 3번(서보) 눌림에 대한 반응 시간차가 컸음

Solution 2. Debouncing 시간 늘림 (40ms -> 200ms)

Problem 3. 회전버튼 동작 issue

정지상태에서 회전버튼을 눌렀을 때, 정지상태여야하는데 동작

Solution 3. rotationFlag 변수 추가

Flag=1인 상태에서만 동작하도록 수정

7. 문제 해결 사례

Problem 4. 회전버튼과 연결된 LED shift task issue

서보가 동작 중일 때 주기적으로 실행되는 LED 쉬프트 태스크가 버튼 1·2에서 설정한 LED 고정 상태를 계속 덮어써서 LED가 모두 켜지거나 꺼지는 현상이 발생

Solution 4. 버튼 입력에 따라 고정 모드와 쉬프트 모드 분리

LED 상태를 서보 동작 여부가 아닌 별도의 LED 모드 상태 변수로 관리

Problem 5. LED 선 issue

다 조립 후 전선 장력이 세서 LED 선이 빠짐

Solution 5. LED와 전선을 글루건으로 고정

8. 개선점 및 확장 아이디어

Idea 1. 타이머 시간 추가

현재는 선풍기가 약 9초간 동작 후 자동 정지하도록 구현되어 있다.

향후에는 타이머 시간을 추가하여 집중 시간과 휴식 시간을 주기로 반복 동작하는 **인터벌 방식**으로 개선할 수 있다.

Idea 2. UART 기능 추가

현재는 버튼 입력과 UART가 같은 동작으로 구현되어 있다.

향후에는 **UART를 통해 시간을 입력받아 타이머 동작 시간을 설정**하는 등의 기능을 추가한다면 편의성 부분을 개선할 수 있을 것이다.

Idea 3. 모서리 모양

현재는 각진 모양으로 구현되어 있다.

향후에는 외형 부분에서, 모서리가 **곡선형태**가 된다면 **미적 / 촉감적**으로 더 좋아질 것이다.

Q & A